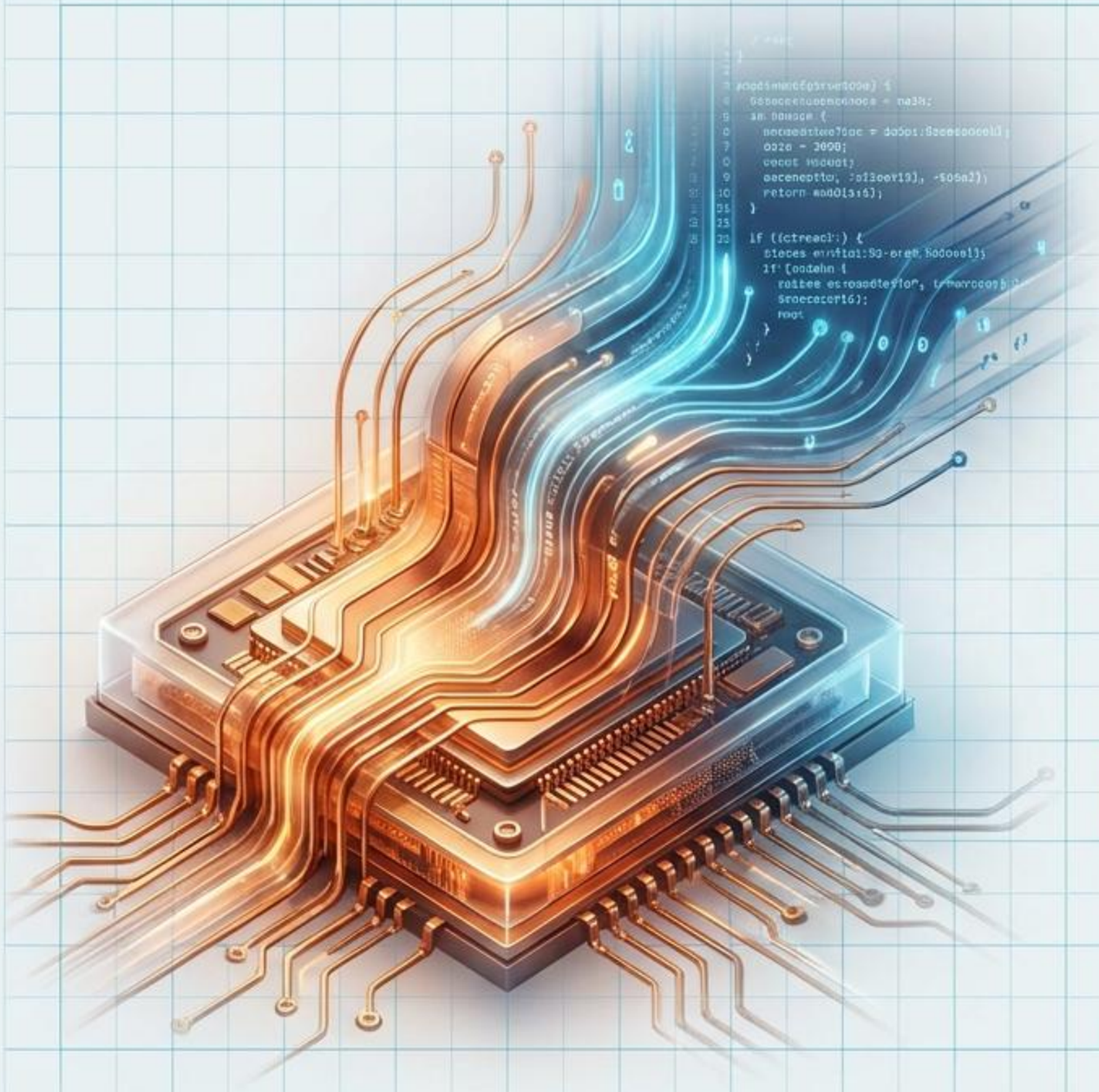


Device Drivers

A Interface Definitiva entre Software e Hardware

Disciplina: Sistemas Operacionais
Curso: Bacharelado em Ciência da Computação
Professor: Prof. Me. Juvenal Neto
Instituição: Universidade do Estado de Mato Grosso (UNEMAT) - Campus de Alto Araguaia

Gemini Notebook

A Situação Cotidiana

Você conecta um dispositivo totalmente novo. Em milissegundos, o **Sistema Operacional** "sabe" exatamente o que fazer.



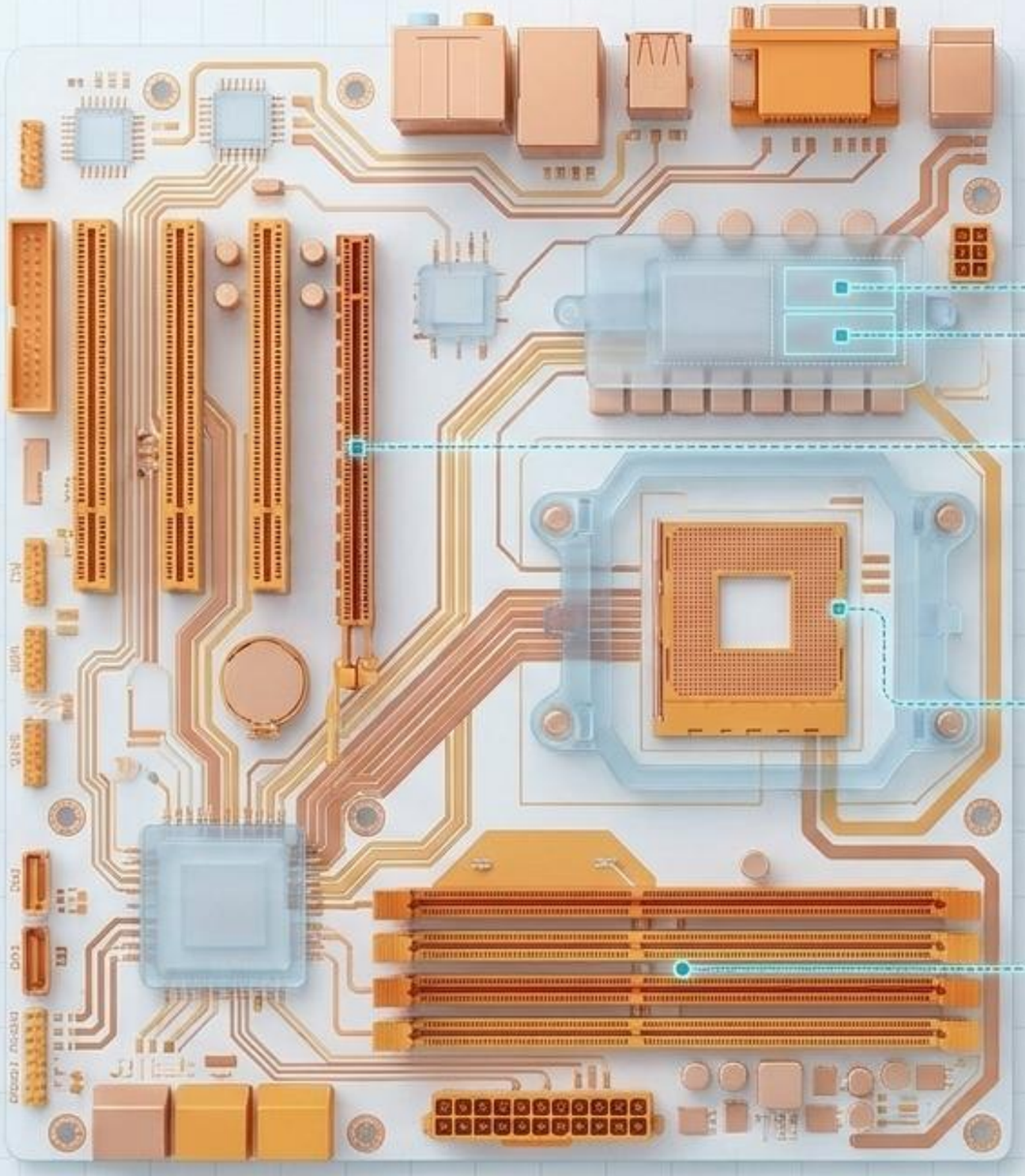
O Problema de Babel

Como um Sistema Operacional, projetado e compilado anos atrás, consegue reconhecer, configurar e operar um hardware com tecnologia que acabou de ser inventada?

A Resposta

Ele não consegue.
Ele precisa de um tradutor.

Gemini Notebook



Controlador (Cérebro Local)

O chip no dispositivo ou na placa-mãe responsável por executar fisicamente as operações elétricas e mecânicas.

Registradores (Interface)

Pequenos espaços de memória no controlador. Onde lemos status ou escrevemos comandos diretos (Ex: bit de busy ou ready).

Barramentos (A Rodovia)

Canais de comunicação (PCI, USB, SATA) por onde trafegam comandos e dados entre a CPU e o controlador.

Interrupções (O Alarme, IRQ)

Sinal de hardware enviado à CPU avisando que o dispositivo terminou uma tarefa ou precisa de atenção imediata.

DMA (O Atalho)

Direct Memory Access. Permite que o controlador transferira grandes blocos de dados direto para a RAM, sem gastar ciclos da CPU.

Gemini Notebook

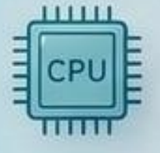
DRIVER



Natureza
Componente de Software (Código).



Localização
Executa na memória principal (RAM), geralmente dentro do Kernel do SO.



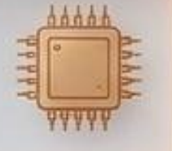
Papel
Traduz chamadas abstratas do SO em instruções binárias específicas.



Tipologia
Pode ser Genérico (ex: teclados padrão) ou Específico (fornecido pelo fabricante).

CONTROLADOR

Natureza
Componente de Hardware (Chips / Circuitos).



Localização
Fisicamente na placa-mãe ou embutido no próprio dispositivo (ex: placa lógica do HD).



Papel
Movimenta motores, lê sensores, opera sinais elétricos e gerencia o buffer local.



Inteligência
Roda firmware próprio, operando de forma independente da CPU principal.



Gemini Notebook



Gemini Notebook



Gemini Notebook

Operações de E/S Assíncronas: Escrita e Leitura (Flowchart)

Operação de Escrita (Write)

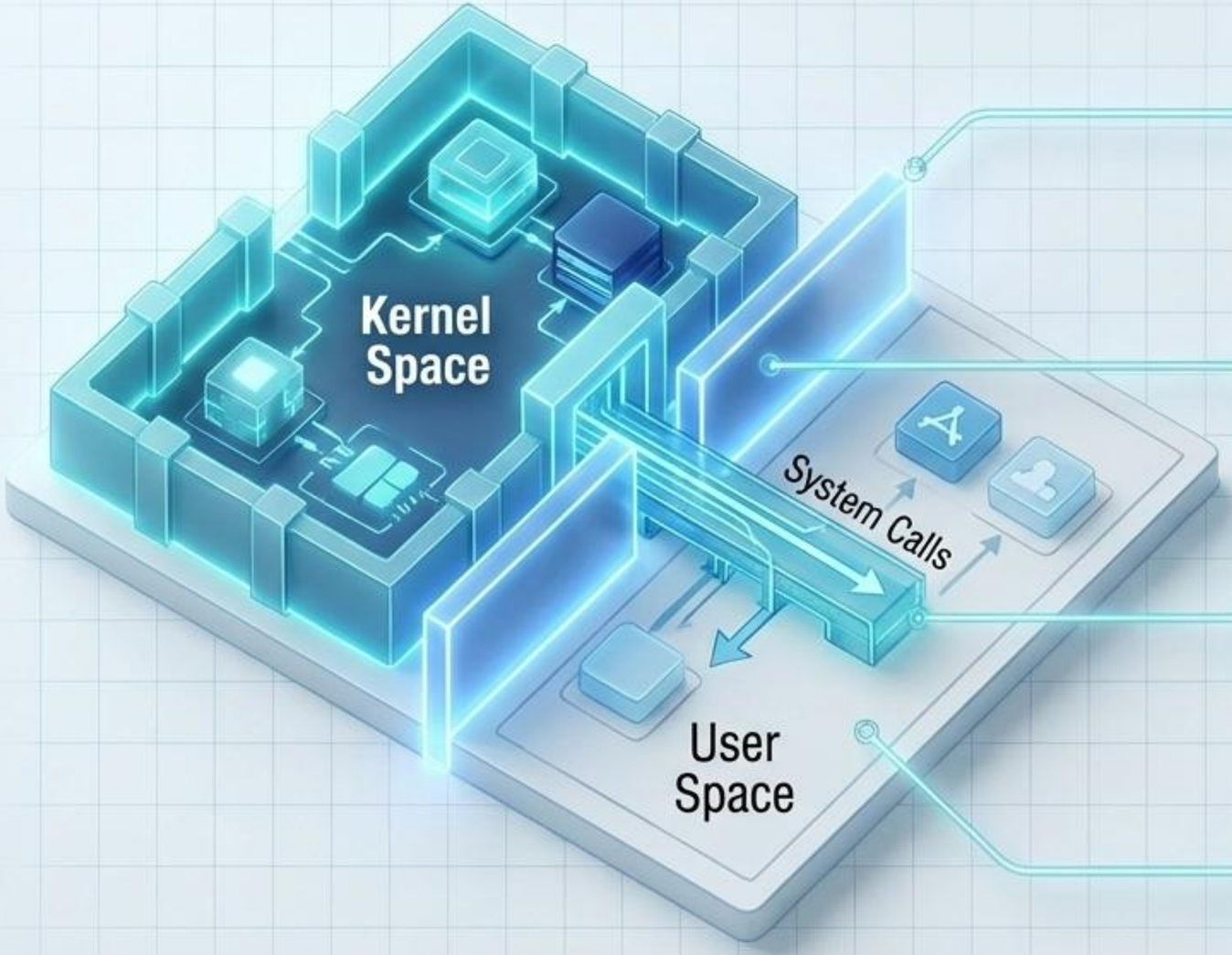
1. App invoca `write()`.
   Aplicação solicita gravação de dados.
2. SO verifica permissões e copia dados do Espaço do Usuário para o Kernel.
   Sistema Operacional gere a transição e buffer.
3. Driver traduz e escreve dados nos registtradores do Controlador.
   Driver converte comando lógico para hardware.
4. Controlador desce os dados para o meio físico.
   Controlador envia sinais ao dispositivo físico.

Operação de Leitura (Read)

1. Meio físico capta dados (ex: clique no mouse).
   Dispositivo físico detecta input.
2. Controlador dispara Interrupção (IRQ) e envia via DMA para a RAM.
   Controlador notifica o sistema e move dados.
3. Driver (Interrupt Handler) reconhece a conclusão e notifica o SO.
   Driver processa a interrupção.
4. SO acorda a Aplicação e libera os dados lidos.
   Sistema Operacional disponibiliza dados para a aplicação.

Gemini Notebook

Arquitetura de Drivers: Kernel Space e User Space



A Muralha (Separação via MMU)

Aplicações rodam em Nível Usuário, confinadas. Drivers precisam acessar portas físicas e DMA, logo, rodam em Modo Privilegiado (Kernel Space).

O Risco de Arquitetura

Um driver no Kernel tem acesso irrestrito à memória. Um bug não trava apenas o aplicativo, causa o colapso do sistema inteiro (Kernel Panic).

O Vetor de Ataque

Códigos maliciosos mascarados de drivers podem contornar totalmente as políticas de segurança do Sistema Operacional.

A Exceção Moderna

Buscando estabilidade, arquiteturas como UMDF (Windows) e FUSE (Linux) permitem que certos drivers não-críticos operem no Espaço do Usuário.

Gemini Notebook



O Paradigma Modular

Nem todo módulo é driver

Protocolos de rede (IPv6) e Sistemas de Arquivos (FAT32) podem ser módulos carregáveis, mas não gerenciam hardwares físicos.

Nem todo driver é módulo

Drivers críticos para o boot (ex: controlador SATA principal) costumam ser compilados estaticamente e embutidos direto no Kernel base.

Ciclo de Vida de um Módulo Dinâmico

1. Carregamento

Binário é copiado para a memória do Kernel.

2. Registro

O código se anuncia para o SO ('Eu sei controlar mouses USB').

3. Utilização

Demandado pelas aplicações em tempo de execução.

4. Remoção

Descarregado dinamicamente para liberar recursos quando ocioso.

Gemini Notebook



A Ilusão POSIX

Para as aplicações, TUDO no Linux/UNIX é um arquivo. O subsistema de E/S abstrai o caos do hardware criando uma Interface Uniforme.

As 5 Chamadas Universais

- **open():** Conecta ao dispositivo.
- **read() / write():** Troca de fluxo de bytes.
- **close():** Libera o acesso.
- **ioctl():** 'Porta dos fundos' para comandos específicos fora do padrão (ex: ejetar gaveta de CD).

O Diretório /dev

Onde dispositivos vivem como nós virtuais. Separados em dispositivos de Bloco (armazenamento endereçável) e Caractere (fluxos contínuos de bytes).

Gemini Notebook

A Jornada de um Pendrive



Conexão Física

Pendrive inserido. Controlador USB percebe variação elétrica, dispara IRQ e notifica o barramento.

Detecção de Classe

O barramento lê o ID do firmware e informa ao SO: “Temos um Dispositivo de Armazenamento de Massa”.

Associação ao Driver

SO vincula o módulo específico (usb-storage). Ocorre o Reconhecimento do Hardware.

Disponibilização Uniforme

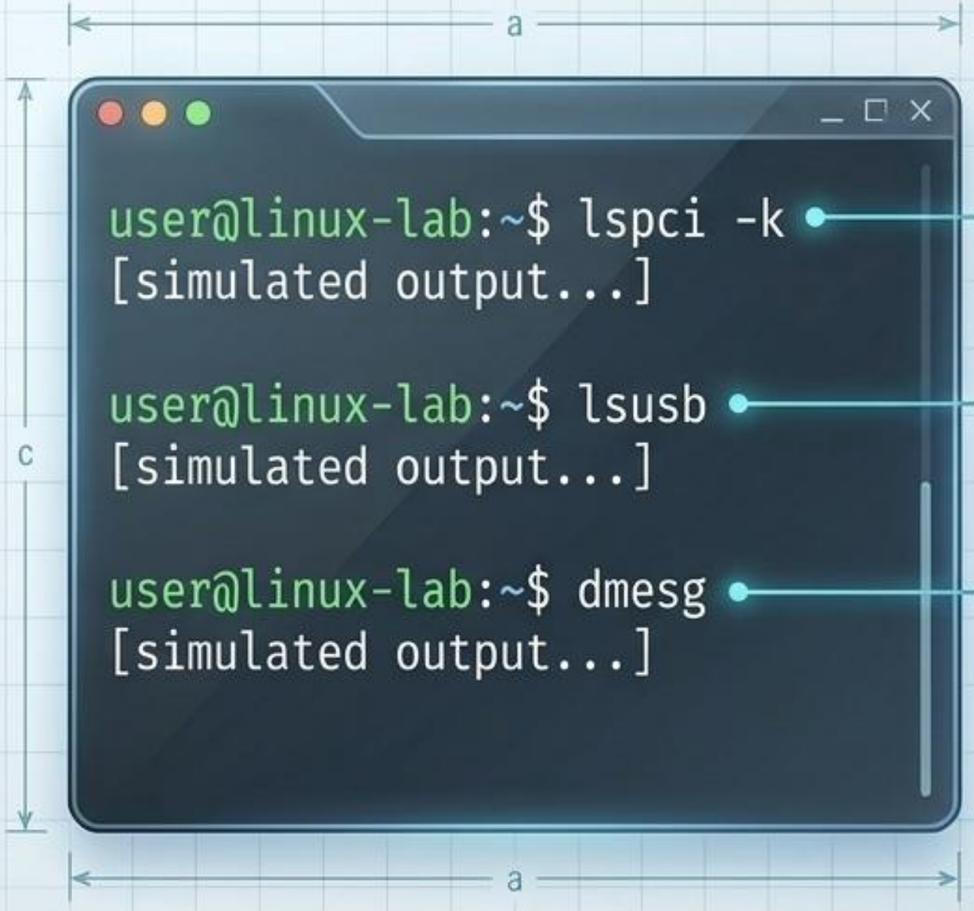
O driver cria a interface no espaço de usuário, gerando o arquivo mágico em `/dev/sdb1`.

Montagem e Acesso

O driver do Sistema de Arquivos (vfat) traduz blocos brutos em pastas e arquivos visíveis à Aplicação.

Gemini Notebook

Laboratório Linux: Diagnóstico de Hardware



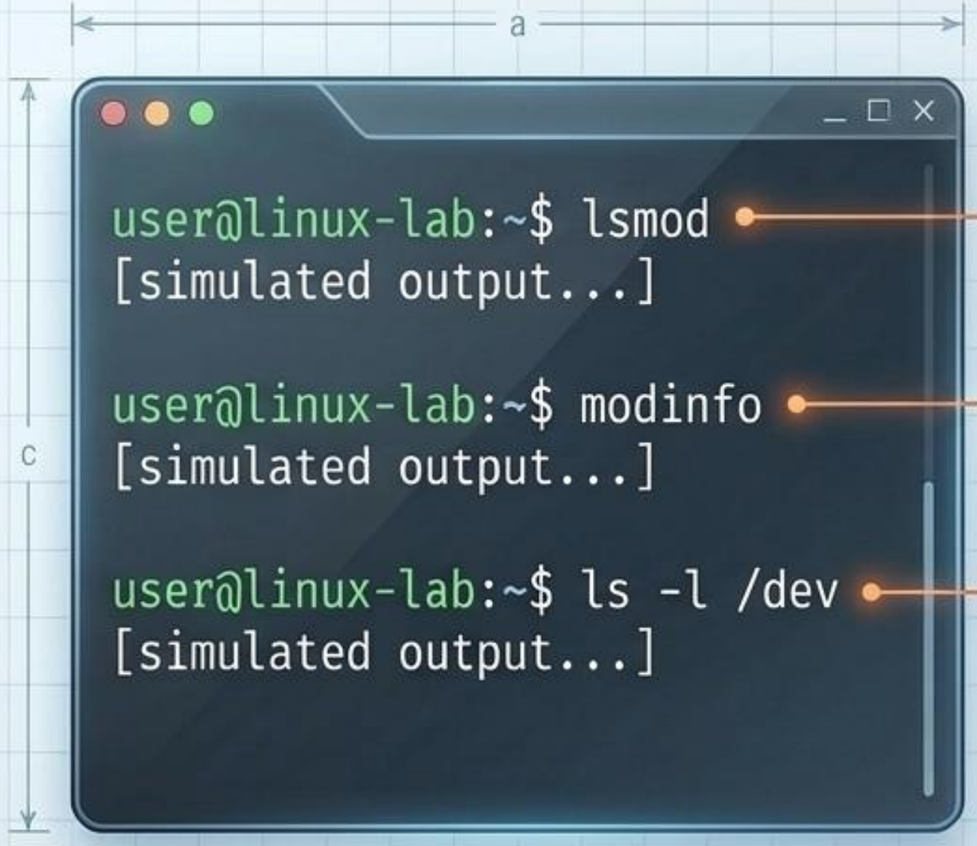
Comando: $ lspci -k
Explicação: Lista os dispositivos conectados aos barramentos PCI/PCIe e revela exatamente qual driver do Kernel está em uso para cada um (Ex: Kernel driver in use: iwlwifi).

Comando: $ lsusb
Explicação: Exibe todos os dispositivos físicos e controladores conectados aos barramentos USB da placa-mãe.

Comando: $ dmesg
Explicação: Exibe o anel de mensagens do Kernel (ring buffer). É o comando essencial para ler logs de boot, rastrear detecção de novo hardware e investigar erros disparados por drivers em tempo real.

Gemini Notebook

Laboratório Linux: Módulos e Dispositivos



Comando: $ lsmod
Explicação: Lista dinamicamente quais módulos e drivers estão carregados na memória do Kernel no momento, exibindo tamanho e dependências.

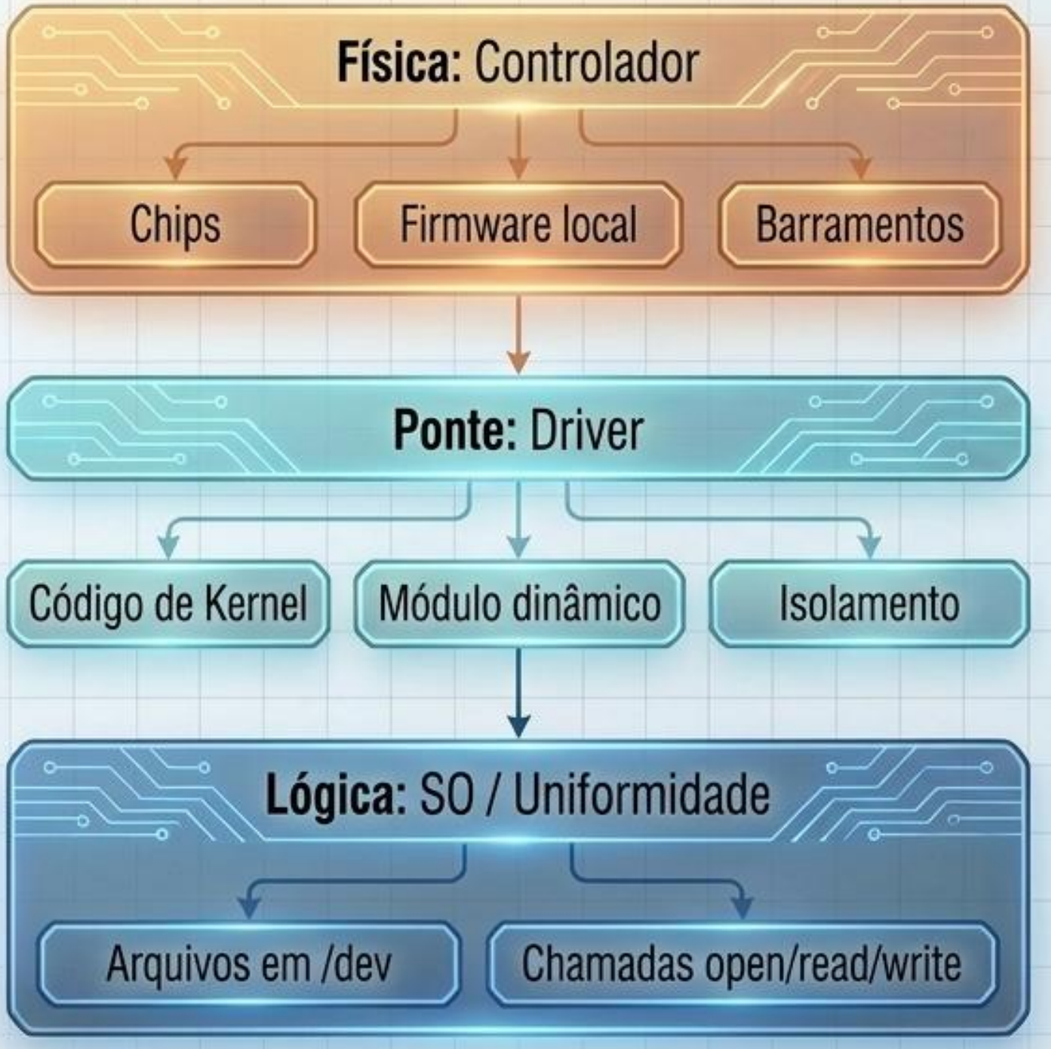
Comando: $ modinfo [nome_do_módulo]
Explicação: Revela os metadados técnicos incorporados no binário do driver (autor, versão, licença e parâmetros aceitos).

Comando: $ ls -l /dev
Explicação: Inspeciona os arquivos mágicos. O primeiro caractere da saída indica a classe gerada pelo driver: 'b' para dispositivos de bloco e 'c' para caractere.

⚠ **ALERTA DE SISTEMA:** Comandos de remoção como 'rmmod' ou 'modprobe -r' forçam a expulsão do módulo do Kernel. Se o driver gerenciar um hardware crítico em uso ativo (rede, disco), o resultado imediato será corrupção de dados ou congelamento total do SO.

Gemini Notebook

Mapa Mental: A Trindade da E/S



Questões Conceituais (Nível Bacharelado)

Concorrência
Se um driver congela esperando uma operação síncrona de I/O em um hardware lento, o que acontece com a CPU e com os processos do Sistema Operacional?

Arquitetura
Quais as vantagens técnicas de compilar um driver estaticamente direto na imagem do Kernel monolítico *versus* mantê-lo como um módulo dinâmico?

Segurança
Como a falha de um programador em validar um ponteiro repassado por `ioctl()` dentro de um driver pode comprometer toda a barreira de segurança do SO?

Gemini Notebook

Atividade Prática de Fixação

Objetivo: *Identificar dinamicamente o fluxo de carregamento de um driver USB em um sistema operacional real.*

- [ ] **Passo 1:** Abra um emulador de terminal em uma distribuição Linux.
- [ ] **Passo 2:** Execute o comando **`'sudo dmesg -w'`** para monitorar o anel de mensagens do Kernel em tempo real.
- [ ] **Passo 3:** Conecte fisicamente um hardware periférico simples e inédito à máquina (ex: um mouse USB genérico).
- [ ] **Passo 4:** Observe as linhas geradas no terminal. Identifique qual arquivo mágico no diretório **`/dev/`** foi instanciado e qual driver assumiu a interface.
- [ ] **Passo 5:** Cancele o log com Ctrl+C. Utilize o comando **`'lsmod | grep [nome_do_driver]'`** para comprovar que o módulo acaba de ser alocado na RAM.

Gemini Notebook